

EDUGENIE – GOOGLE GEMINI POWERED LEARNING ASSISTANT

SYSTEM DESIGN DOCUMENT

1. Introduction

EduGenie is a Generative AI-powered educational web application designed to assist students with learning, revision, and understanding of academic concepts.

The system provides five major learning functions:

1. Question and Answer
2. Concept Explanation
3. Quiz Generation
4. Text Summarization
5. Personalized Learning Recommendations

The application follows a lightweight client-server architecture. A browser-based frontend communicates with a FastAPI backend, which processes requests and interacts with Google Gemini for AI-generated educational responses. An optional local LaMini-Flan-T5-783M model can be used for concept explanations.

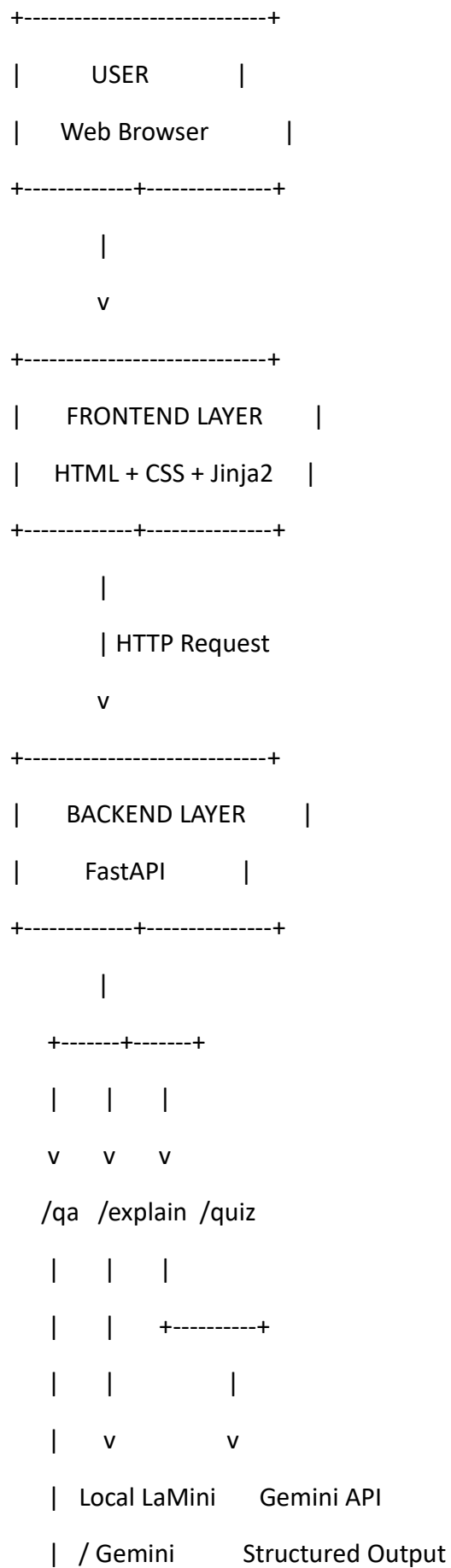
2. Design Objectives

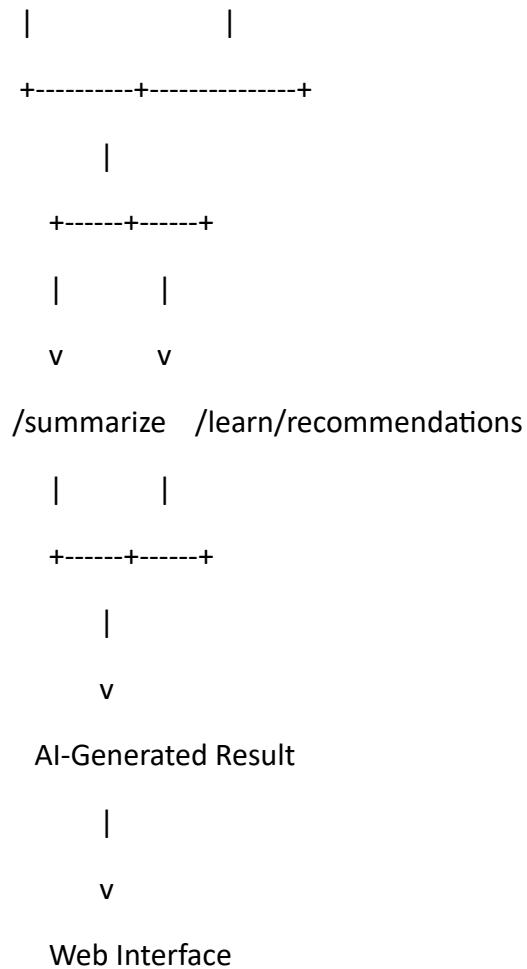
The major objectives of the system design are:

- Provide a simple and user-friendly learning interface.
 - Separate frontend, backend, and AI processing responsibilities.
 - Provide independent API endpoints for different learning functions.
 - Integrate Generative AI into educational workflows.
 - Support structured quiz generation.
 - Provide optional local AI processing for concept explanations.
 - Make the application easy to test and maintain.
 - Allow future expansion with additional educational features.
-

3. Overall System Architecture

The high-level architecture consists of the following layers:





4. Architectural Layers

The presentation layer is responsible for interaction between the student and EduGenie.

- HTML
- CSS
- Jinja2 templates
- Web browser

- Display the EduGenie interface.

- Accept user input.
- Provide feature selection.
- Submit requests.
- Display AI-generated responses.

The frontend is intentionally lightweight so that students can interact with the system without requiring a complicated application interface.

5. Application Layer

The application layer is implemented using FastAPI.

Technology

FastAPI

Responsibilities

- Receive HTTP requests.
- Validate incoming data.
- Route requests to the appropriate functionality.
- Communicate with AI services.
- Process AI responses.
- Return results to the frontend.
- Provide API documentation.

FastAPI also provides the interactive API documentation available through `/docs`.

6. AI Processing Layer

The AI processing layer provides the intelligence of EduGenie.

The primary AI service is Google's Gemini API.

Gemini is used for:

- Question answering.
- Concept explanation fallback.
- Quiz generation.

- Text summarization.
- Learning recommendations.

The current implementation uses a configurable Gemini model with gemini-2.5-flash as the default model.

7. Optional Local AI Layer

EduGenie also supports an optional local explanation model:

LaMini-Flan-T5-783M

This model can be used for concept explanation when local-model support is enabled.

The explanation architecture can therefore be represented as:

User requests explanation

|

v

/explain endpoint

|

v

Local model enabled?

/ \

Yes No

| |

v v

LaMini Gemini

\ /

\ /

v v

Explanation

If the local model is unavailable or disabled, Gemini can act as the explanation fallback. The repository specifies that local-model execution may be slower and more memory-intensive on CPU, so it is disabled by default.

8. API Design

EduGenie provides separate API endpoints for its major educational functions.

8.1 Question Answering

Endpoint:

/qa

Input

A user's academic question.

Processing

The question is sent to Gemini.

Output

An AI-generated answer.

Flow

Question

↓

/qa

↓

Gemini

↓

Answer

9. Concept Explanation Design

Endpoint:

/explain

Input

- Topic
- Learning level

Processing

The application can use the optional local LaMini model or Gemini.

Output

A learner-appropriate explanation.

Example

Topic: Pythagoras Theorem

Level: Beginner

Flow

Topic + Level



/explain



Local Model / Gemini



Explanation

10. Quiz Generation Design

Endpoint:

/quiz

The quiz module accepts educational content and generates three multiple-choice questions.

Input

A topic or educational passage.

Processing

Gemini generates structured quiz information.

Output

- Three questions.
- Answer options.
- Correct answers.

Flow

Educational Content



/quiz



Gemini



Structured Output



3 MCQs + Answers

The current project specifically uses Gemini structured output for quiz generation.

11. Text Summarization Design

Endpoint:

/summarize

Input

A passage or educational text.

Processing

The text is sent to Gemini for summarization.

Output

A concise summary containing the important information.

Flow

Long Passage



/summarize



Gemini



Concise Summary

12. Learning Recommendation Design

Endpoint:

/learn/recommendations

Input

- Subject/topic
- Learning level

Processing

Gemini analyzes the requested subject and learner level.

Output

A recommended learning sequence.

Flow

Subject + Level

↓

/learn/recommendations

↓

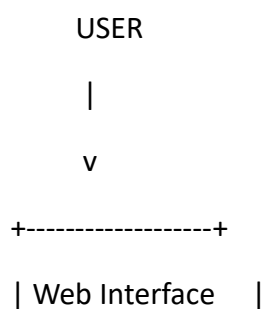
Gemini

↓

Learning Recommendations

13. Data Flow Design

The general data flow is:



| HTML/CSS/Jinja2 |

+-----+-----+

|

| HTTP Request

v

+-----+-----+

| FastAPI Backend |

+-----+-----+

|

+-----+-----+

| | |

v v v

/qa /explain /quiz

| | |

| | +----> Gemini

| |

| +-----> LaMini / Gemini

|

+-----> Gemini

/summarize -----> Gemini

/learn/recommendations -> Gemini

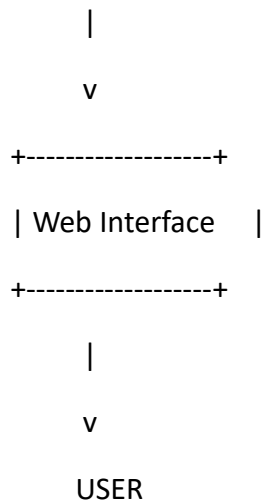
|

v

+-----+-----+

| Generated Result |

+-----+-----+



14. Component Design

The major system components are:

Component	Responsibility
Web Browser	User interaction
HTML Templates	Interface structure
CSS	Interface styling
Jinja2	Dynamic template rendering
FastAPI	Backend/API framework
Uvicorn	Application server
Q&A Module	Answers user questions
Explanation Module	Explains concepts
Quiz Module	Generates MCQs
Summary Module	Summarizes passages
Learning Module	Generates learning recommendations
Gemini	Main Generative AI service
LaMini	Optional local explanation model
Pytest	Automated testing

15. Request Processing Design

Every user operation follows a common processing pattern.

Step 1 – User Input

The student selects a feature and enters the required content.

Step 2 – HTTP Request

The frontend sends the request to the corresponding FastAPI endpoint.

Step 3 – Request Validation

The backend validates the request information.

Step 4 – Feature Processing

The selected module processes the request.

Step 5 – AI Processing

The request is sent to Gemini or, where configured, the local explanation model.

Step 6 – Response Generation

The AI model generates the required educational output.

Step 7 – Response Delivery

The backend returns the result.

Step 8 – Display

The frontend displays the generated content to the student.

16. Error Handling Design

The system should handle possible errors during request processing.

Potential errors include:

- Empty input.
- Invalid request data.
- Gemini API errors.
- Network errors.
- Model-loading errors.

- Invalid AI output.
- Server-side exceptions.

The backend should return an appropriate error response instead of terminating the application.

For local-model operation, the system can fall back to Gemini when appropriate.

17. Configuration Design

Sensitive configuration information is separated from the source code.

The Gemini API key is stored through environment configuration.

Example:

```
GEMINI_API_KEY=your_api_key_here
```

The .env file should not be committed to the Git repository.

This design prevents sensitive credentials from being directly embedded in application source code.

18. Model Configuration

The Gemini model is configurable rather than permanently hard-coded to a single model.

The current repository uses:

Default Gemini Model:

```
gemini-2.5-flash
```

This allows the project to adapt to future Gemini model changes without redesigning the complete application.

19. Testing Design

Testing is incorporated into the project to verify backend functionality.

Testing Framework

Pytest

The project uses mocked Gemini responses so that tests can be executed without requiring paid Gemini API calls.

The test suite can be executed using:

```
pytest -q
```

This approach separates application testing from dependence on live AI service responses.

20. Health Monitoring Design

A health-check endpoint is provided:

```
/health
```

This endpoint can be used to verify whether the application is running correctly.

The health-check mechanism can later be extended for production monitoring systems.

21. Security Design

The current design follows basic security practices:

- API keys are stored through environment variables.
- Secrets are excluded from source control.
- User inputs are processed through backend endpoints.
- Production deployment should use HTTPS.
- Rate limiting should be added for public deployments.
- Authentication should be introduced if user-specific data is stored.

The current application is intentionally lightweight, while the repository identifies authentication, rate limiting, logging, secret management, request-size limits, HTTPS, and persistent storage as production enhancements.

22. User Interface Design

The interface is designed around simplicity.

A typical interaction consists of:

```
+-----+
|      EduGenie      |
| Google Gemini Powered Learning  |
|      Assistant      |
```

```

+-----+
| Select Learning Function      |
|                               |
| [ Q&A / Explain / Quiz / Summary ] |
|                               |
| Input:                       |
| +-----+ |
| | Enter question or passage... | |
| |                               | |
| +-----+ |
|                               |
| [Submit]                    |
+-----+
|      Result                |
|                               |
| AI-generated educational response |
|                               |
+-----+

```

The interface should make the main learning functions immediately accessible to the user.

23. Deployment Design

For local development, the application runs using Uvicorn.

```
uvicorn main:app --reload
```

The application is accessed at:

<http://127.0.0.1:8000>

FastAPI documentation:

<http://127.0.0.1:8000/docs>

Health check:

http://127.0.0.1:8000/health

These commands and endpoints are documented in the current repository README.

24. Technology Stack Design

Layer	Technology
Frontend	HTML, CSS
Template Engine	Jinja2
Backend	FastAPI
Application Server	Uvicorn
Primary AI	Google Gemini
Optional Local AI	LaMini-Flan-T5-783M
Programming Language Python	
Testing	Pytest
Configuration	Environment Variables
API Documentation	FastAPI / Swagger UI

25. Design Principles

The system follows these design principles:

Simplicity

The interface and architecture are intentionally lightweight.

Modularity

Each educational function is separated into its own processing module.

Reusability

The FastAPI architecture allows the same AI services to support different educational tasks.

Maintainability

Separating frontend, backend, AI processing, configuration, and testing makes the application easier to maintain.

Extensibility

The architecture allows future features such as user accounts, databases, progress tracking, adaptive learning, and additional AI capabilities.

26. Future Design Enhancements

Future versions can extend the design with:

26.1 Database Layer

A database could store:

- User accounts.
- Quiz scores.
- Learning history.
- Completed topics.
- Personalized progress.

26.2 Authentication Layer

Authentication can provide:

- Login.
- Registration.
- User-specific learning data.
- Role-based access.

26.3 Adaptive Learning Engine

The system could analyze previous quiz results and dynamically adjust future learning recommendations.

26.4 Multimodal AI

Future versions could support:

- Images.
- PDFs.
- Diagrams.
- Voice input.
- Audio responses.

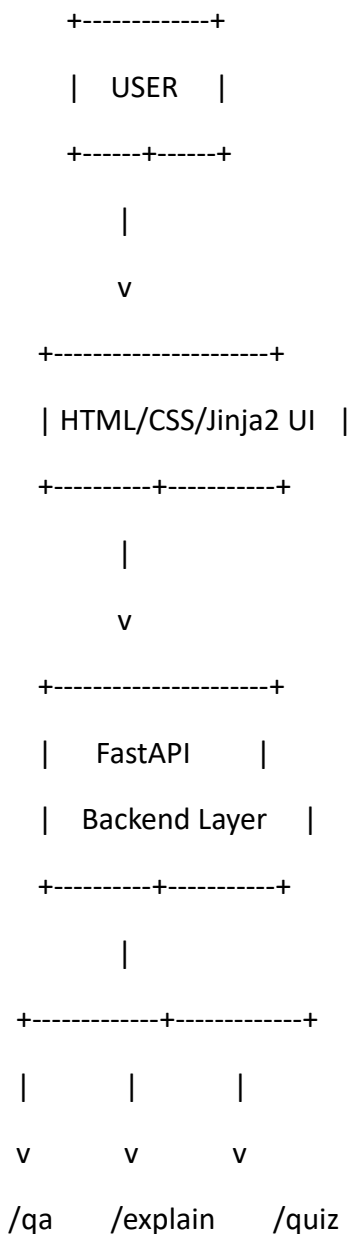
26.5 Analytics Dashboard

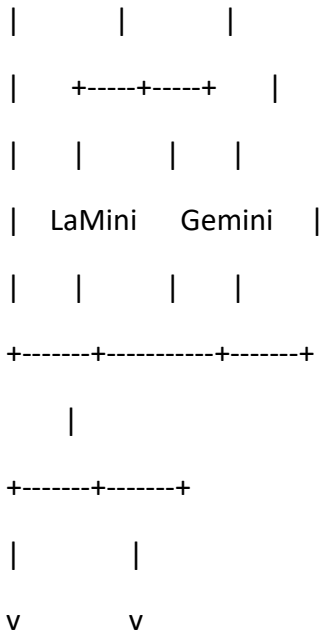
Students could receive:

- Progress charts.
 - Quiz performance.
 - Weak-topic identification.
 - Study recommendations.
-

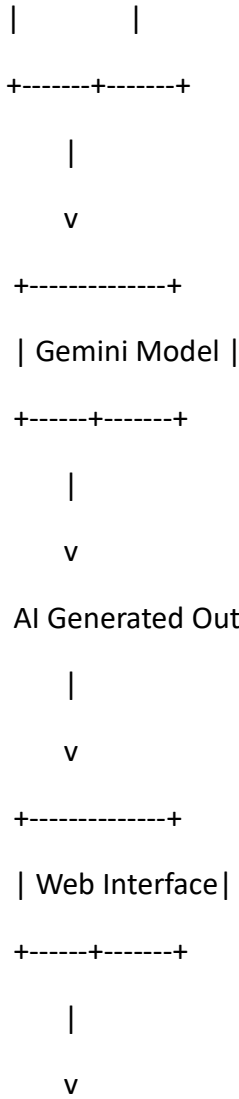
27. Complete System Design Flow

The complete design can be summarized as:





/summarize /learn/recommendations



28. Conclusion

The EduGenie system design provides a modular architecture for an AI-powered educational assistant.

The design separates the user interface, backend processing, AI services, configuration, and testing components. FastAPI acts as the central application layer, while Gemini provides the primary Generative AI capabilities.

The architecture supports five major educational operations:

Q&A → Explanation → Quiz → Summarization → Learning Recommendations

The optional local LaMini model provides an additional processing path for concept explanation.

This design provides a strong foundation for extending EduGenie into a more advanced personalized learning platform with authentication, persistent progress tracking, adaptive learning, analytics, multimodal AI, and other educational capabilities.